

Representación ligada.

Definición de un Nodo

```
Nodo
inicio
    tipo elemento
    Nodo siguiente
fin
```

Definición de una Lista

```
Lista
inicio
    Nodo cabeza
    int longitud
fin
```

Inicializar la lista

```
inicializacion (Lista lista)
inicio
    lista->cabeza ← null
    lista->longitud ← 0
fin
```

Determinar si una lista está vacía

```
int esVacia(Lista lista)
inicio
    si(lista->cabeza=null)
        inicio
            regresa 1
        fin
    otro
        inicio
            regresa 0
        fin
fin
```

Recorrer una lista

```
void imprimir(Lista lista)
inicio
    Nodo nodoActual ← lista->cabeza
    mientras(nodoActual != null)
        inicio
            imprime (lista->elemento)
            aux ← aux->siguiente
        fin
    fin
```

Buscar un elemento en la lista

```
int buscar(Lista lista, tipo buscado)
inicio
    Nodo nodoActual ← lista->cabeza
    posicion ← -1
    i ← 0
    mientras(nodoActual != null)
    inicio
        si(nodoActual ->elemento = buscado)
        inicio
            posicion ← i
            interrumpe
        fin
        otro
        inicio
            nodoActual ← nodoActual ->siguiente
            i+=1
        fin
    fin
    fin
    regresa posicion
fin
```

Insertar en lista vacía

```
insertarListaVacía (Lista lista, tipo nuevoElemento)
inicio
    Nodo nuevoNodo ← nuevo Nodo
    nuevoNodo->elemento ← nuevoElemento
    nuevoNodo->siguiente ← NULL
    lista->cabeza ← nuevoNodo
    lista->longitud ← lista->longitud+1
fin
```

Insertar al inicio de la lista

```
insertarInicio(Lista lista, tipo nuevoElemento)
inicio
    si(esVacía (lista) = 1)
        inicio
            Nodo nuevoNodo ← nuevo Nodo
            nuevoNodo->elemento ← nuevoElemento
            nuevoNodo->siguiente ← NULL
            lista->cabeza ← nuevoNodo
        fin
    otro
        inicio
            Nodo nuevoNodo ← nuevo Nodo
            nuevoNodo->elemento ← nuevoElemento
            nuevoNodo->siguiente ← lista->cabeza
            lista->cabeza ← nuevoNodo
        fin
    lista->longitud ← lista->longitud+1
fin
```

Insertar al final de la lista

```
insertarFinal(Lista lista, tipo nuevoElemento)
inicio
    si(esVacia (lista) = 1)
        inicio
            Nodo nuevoNodo ← nuevo Nodo
            nuevoNodo->elemento ← nuevoElemento
            lista->cabeza ← nuevoNodo
        fin
    otro
        inicio
            Nodo nuevoNodo ← nuevo Nodo
            nuevoNodo->elemento ← nuevoElemento
            nuevoNodo->siguiente ← NULL

            Nodo actual ← lista->cabeza
            mientras(actual ->siguiente !=null)
                inicio
                    actual ← actual ->siguiente
                fin
            actual->siguiente ← nuevoNodo
        fin
    lista->longitud ← lista->longitud+1
fin
```

Borrar el primer elemento

```
eliminarInicio (Lista lista)
inicio
    si(esVacia(lista)=1)
        inicio
            IMPRIMIR La lista esta vacía
        fin
    otro
        inicio
            lista->cabeza ← lista->cabeza->siguiente
            lista.longitud ← lista->longitud-1
        fin
fin
```

Borrar el último elemento

eliminarFinal (Lista lista)

inicio

 si(esVacía (lista) = 1)

 inicio

 IMPRIMIR La lista esta vacía

 fin

 otro

 inicio

 Nodo actual ← lista->cabeza

 i ← 1

 mientras (i < lista->longitud-1)

 inicio

 actual ← actual->siguiente

 i++

 fin

 actual->siguiente ← NULL

 lista->longitud ← lista->longitud-1

 fin

fin